Assessing, Exploiting, and Mitigating Syntactic Robustness Failures in LLM-Based Code Generation

Laboni Sarker
University of California, Santa Barbara
labonisarker@ucsb.edu

Mara Downing
University of California, Santa Barbara
maradowning@ucsb.edu

Achintya Desai
University of California, Santa Barbara
achintya@ucsb.edu

Tevfik Bultan
University of California, Santa Barbara
bultan@ucsb.edu

Abstract

Rapid advances in the field of Large Language Models (LLMs) have made LLM-based code generation an important area for investigation. An LLM-based code generator takes a prompt as input and produces code that implements the requirements specified in the prompt. Many software requirements include mathematical formulas that specify the expected behavior of the code to be generated. Given a code generation prompt that contains a mathematical formula, a reasonable expectation is that, if the formula is syntactically modified without changing its semantics, the generated code for the modified prompt should be semantically equivalent. We formalize this concept as syntactic robustness and investigate the syntactic robustness of LLMs as code generators. Our experimental assessment demonstrates that LLMs are not syntactically robust for code generation prompts with formulas, especially for the ones that require mathematical reasoning. We investigate attack strategies that can further deteriorate the syntactic robustness of LLMs. Finally, to mitigate syntactic robustness failures in LLMs, we propose a pre-processing step that uses reductions to transform formulas in prompts to a simplified form. Our experimental results demonstrate that the syntactic robustness of LLM-based code generation improves significantly using our approach, improving syntactic robustness of LLMs from 54.05% to 74.42%.

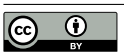
CCS Concepts

• **Software and its engineering → Software verification and validation**.

Keywords

syntactic robustness, LLMs, code generation, exploitation, preprocessing, formula reduction.

ACM Reference Format:
Laboni Sarker, Mara Downing, Achintya Desai, and Tevfik Bultan. 2026. Assessing, Exploiting, and Mitigating Syntactic Robustness Failures in LLM-Based Code Generation . In *2026 IEEE/ACM Third International Conference on AI Foundation Models and Software Engineering (FORGE '26), April 12–13, 2026, Rio de Janeiro, Brazil.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3793655.3793727



This work is licensed under a Creative Commons Attribution 4.0 International License.
FORGE '26, Rio de Janeiro, Brazil
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2477-0/2026/04
https://doi.org/10.1145/3793655.3793727

1 Introduction

Large language models (LLMs) are becoming popular in the software engineering community [1, 2], especially for code generation tasks [3–6], and have shown strong performance on code generation benchmarks [7–9]. Recently, LLMs have also been used for code generation in the scientific computing domain [10–16] and for generating code that efficiently handles complex mathematical computations [14–18]. Furthermore, LLMs have shown promise in handling formal specifications [19, 20], which rely on mathematical formulas in the specification of software engineering tasks. As its adoption grows, there is a critical need for systematic evaluation of the correctness and reliability of LLM-based code generation [1, 5, 6, 21–25].

Reliability in machine learning systems is often assessed through robustness, which has been extensively studied for classification tasks [26–32]. Robustness in classification models involves defining a neighborhood around an input with a known classification and ensuring the same classification is obtained within that neighborhood [33]. For generative models, however, this definition must be adapted, as obtaining and verifying objectively accurate outputs is more complex—many different outputs may be objectively correct. This highlights the need for well-defined robustness metrics to analyze LLM-based code generators, which are a subdomain of generative models.

Several prior works [1, 5, 6, 21–25] highlight the importance of studying robustness for code generation by LLMs, and some do investigate robustness [6, 22, 34]. However, the impact of syntactic transformations of the prompts together with the increasing syntactic distance (Section 4) has not been investigated. Additionally, black-box attacks specifically designed to expose failures in the robustness of LLM-based code generators in handling syntactic variations of semantically equivalent prompts (Section 5) have not been explored.

In this paper, we define a formal robustness measure for LLM-based code generators called syntactic robustness (Section 3) and analyze the performance of LLMs with respect to this measure (Section 8). Intuitively, we define syntactic robustness as the degree

This material is based on research supported by NSF under grants 2124039 and 2008660 and DARPA under the agreement number N66001-22-2-4037. The U.S. Government is authorized to reproduce and distribute reprints for Governmental purposes notwithstanding any copyright notation thereon. The views and conclusions contained herein are those of the authors and should not be interpreted as necessarily representing the official policies or endorsements, either expressed or implied, of the U.S. Government.

to which the semantically equivalent prompts elicit semantically equivalent responses from the LLM code generator.

We define a set of syntactic transformations (Section 4) that generate syntactically different but semantically equivalent variations of mathematical formulas to assess the syntactic robustness of LLM-based code generators. We present attack strategies (Section 5) to exploit the syntactic robustness failures of the existing LLMs while minimizing the modifications to the formulas in the prompts. Furthermore, we present a pre-processing step (Section 6) to mitigate syntactic robustness failures and defend against such attacks, and demonstrate its effectiveness. Our assessment reveals that state-of-the-art LLMs lack syntactic robustness and that prompts requiring mathematical reasoning exhibit lower robustness compared to those involving direct translation of mathematical formulas into code. Our experimental evaluation also shows that our attacks can effectively exploit the syntactic robustness failures of the LLMs. Finally, our experiments demonstrate that our pre-processing effectively mitigates syntactic robustness failures, significantly improving syntactic robustness of LLMs.

Contributions. Our contributions in this paper are as follows:

(1) A formal definition of syntactic robustness (Section 3).
(2) A set of prompts with mathematical formulas, designed to benchmark the robustness of LLM-based code generators (provided as artifact [35]).
(3) A set of rules to mutate the mathematical formulas, that along with the original prompts, can be used for syntactic robustness evaluation (Section 4).
(4) A set of black-box attacks for exploiting syntactic robustness failures of LLMs (Section 5).
(5) A prompt pre-processing technique that significantly helps in mitigating the syntactic robustness failures of LLMs (Section 6).
(6) Empirical assessment of syntactic robustness, along with novel attack and mitigation techniques, on state-of-the-art LLM foundational models and frameworks (Section 8).

Organization. We discuss motivating examples of code generation prompts in Section 2, then we provide a formal definition of syntactic robustness and related concepts in Section 3. We discuss our formula transformation procedure in Section 4 and detail our attacks in Section 5. We explain our pre-processing procedure in Section 6, and then explicate implementation design details in Section 7. We present our experimental evaluation in Section 8, discuss the related work in Section 9, and conclude the paper in Section 10.

2 Motivation

Mathematical formulas and constraints are common in code generation prompts in a variety of domains [10, 11, 17, 18, 20]. Given many equivalent ways one can write a mathematical formula or constraint, ensuring syntactic robustness is crucial for code generation in such contexts. Thus, it is critical to evaluate the robustness of LLM-based code generators for prompts with mathematical formulas. Note that, the equivalence of mathematical formulas can be defined clearly and unambiguously, allowing for an automated evaluation of robustness [36–38]. This is in contrast to prior works that mutate the text of the prompt [5, 6], which can alter the meaning due to the ambiguity of natural language.

Prompt A *"Implement a C program which takes 'a','b' as inputs*
where method func(a: array<int>, b: array<int>)
returns (result: bool)
requires a != null && b != null
ensures result ==> exists i, j :: 0 <= i < a.Length &&
0 <= j < b.Length && ***Eq***
ensures !result ==> forall i, j :: 0 <= i < a.Length &&
0 <= j < b.Length ==> a[i] != b[j]."

Expressions for placeholder *Eq* in Prompt A
Eq: a[i] == b[j]
Eq: (a[i]) * 7 - (b[j]) * 7 == (b[j]) * 7 - (b[j]) * 7

Gpt-4o Response Snippet for *Eq*: *a[i] == b[j]*

```
...
    for (int j = 0; j < length; j++) {
        if (a[i] == b[j]) {
            return true; }}}
return false; }
```

Gpt-4o Response Snippet for *Eq*: *(a[i]) * 7 - (b[j]) * 7 == (b[j]) * 7 - (b[j]) * 7*

```
...
    if ((a[i]*7 - b[j]*7) == (b[j]*7 - b[j]*7)){
            return true; }}}
... if (a[i] == b[j]) { return false; } ...
  return true; }
```

Figure 1: Code segments from the programs generated by Gpt-4o for Prompt A

Syntactically transformed prompts with mutated formulas can also be interpreted as adversarial samples for assessing the robustness of LLM-based code generators under adversarial attacks [39–41]. Our evaluation demonstrates that LLMs are vulnerable to adversarial prompt-based attacks, motivating the need for assessing and improving the syntactic robustness of LLM-based code generators.

Example prompts. We motivate our work through a series of example code generation prompts involving mathematical formulas from different datasets. Prompt A is from the clover dataset [20], which contains formal specifications, Prompt B is generated from a HackerRank problem [42], and Prompt C and D are from the MaTT dataset [43].

Prompt B *"The number of digits present in a number is represented by 'd'. 'a' represents a natural number. Write a C code which will return all possible 'x' given a value 'b' which will satisfy the following pre and post-condition. Pre-condition: 1 <= b && b <= 19 Post condition: d=b and* ***Eq****. Print only the value of all possible values of x in comma separated form."*

Expressions for placeholder *Eq* in Prompt B
Eq: x = a^b
Eq: x+b = a^b+b

Prompt A and B demonstrate the syntactic robustness challenges in LLM-based code generation. Prompt A shows a code generation query to generate a program that returns *True* if two input arrays share any common element and *False* otherwise. On the other hand, Prompt B asks to generate a program that can find all b-digit

Gpt-4o Response Snippet for *Eq*: x=a^b

```
while (1) {
      long long x = pow(a, b);
```

Gpt-4o Response Snippet for *Eq*: x+b=a^b+b

```
for (int a = 1; ; a++) {
       long long aPowerB = pow(a, b);
       long long x = aPowerB + b;
```

Figure 2: Code segments from the programs generated by Gpt-4o for Prompt B

numbers that are also the b-th power of a natural number. The placeholder ***Eq*** replacements for both Prompt A and B are shown at the bottom of each prompt and the corresponding outputs are presented in Figures 1 and 2, respectively.

Figures 1 and 2 show code segments generated by GPT-4o highlighting the differences for semantically equivalent prompts. Despite the semantic equivalence of the mathematical formulas in the prompts, the generated codes reflect different interpretations. Both examples demonstrate the case where Gpt-4o struggles to handle syntactic mutations on a simple equation. Note that, in both prompts, the natural language texts are kept the same—the only distinction is the semantically equivalent but syntactically different equations. However, the generated codes from Gpt-4o are not semantically equivalent. In our experiments (Section 8), we show that this problem exists across multiple LLMs for a variety of prompts.

In code generation tasks, semantically equivalent formulas can often appear in syntactically different forms, particularly when derived from complex specifications. As the examples above demonstrate, enhancing the reliability of LLMs requires investigating and improving their syntactic robustness before deploying them for such tasks.

Reasoning and Translation Prompts. We define two types of prompts for our analysis: reasoning and translation prompts. Consider the Prompt C below which asks for a program to find out the work done when a body is moved from one fixed position to another. The expected output from this prompt is to get the amount of work done which is $9.8 \times a \times b \times (1/c - 1/e)$ obtained by computing the integral $\int_c^e (9.8 \times a \times b/d^2)$ with respect to d.

> **Prompt C** *"Newton's Law of Gravitation states that two bodies with masses a and b attract each other with a force F=9.8 * a * b / d^2 where d is the distance between the bodies and 9.8 is the gravitational constant. If one of the bodies is fixed, implement a C program to find the work needed to move the other from d=c to d=e where 'a', 'b','c','e' are inputs to the program. Assume none of 'a', 'b', 'c', 'd' or 'e' are 0. Print only the work needed up to 2 digit precision after decimal (do not print anything else)."*

We create a variation of Prompt C (we call Prompt D) by replacing the second sentence of the prompt as *"Implement a C program to find the attraction force where 'a', 'b' and 'd' are inputs to the program."* Prompt D contains the same equation as Prompt C but asks to calculate the attraction force given the equation, where the expected output is computed by evaluating the expression $9.8 \times a \times b/d^2$, which is provided in the prompt.

Although both prompts involve the same mathematical formula, Prompt C requires mathematical reasoning to perform integration, while Prompt D requires translation of the input formula to an expression in the target programming language to be evaluated during program execution. We name these two classes of prompts as *reasoning* and *translation* prompts, respectively. Our experiments indicate that syntactic robustness of LLM-based code generators differ for reasoning and translation prompts.

3 Syntactic Robustness

In this section, we formalize the concept of syntactic robustness. We start by defining key concepts including LLM-based code generators, prompts, and generated programs and their equivalence, before introducing our formal definition of syntactic robustness.

LLM-based Code Generators. We define an *LLM-based code generator* as follows:

Definition 1. *An* **LLM-based code generator** G *takes a prompt* $P \in \mathcal{P}$ *as input and generates code* $C \in \mathcal{C}$*, denoted as* $G : \mathcal{P} \to \mathcal{C}$*, where* $\mathcal{P}$ *is the set of prompts and* $\mathcal{C}$ *is the set of programs.*

In this paper, we focus on prompts that contain both English text and mathematical formulas:

Definition 2. *A* **prompt** $P \in \mathcal{P}$ *consists of an English text* T *and a mathematical formula* F *denoted as a tuple* $P\langle T, F\rangle$.

When an LLM-based code generator generates code C for prompt $P\langle T, F\rangle$, we denote it as: $G(P\langle T, F\rangle) = C$.

Semantic Equivalence of Formulas. Two syntactically different formulas F_1 and F_2 can be semantically equivalent. We denote the semantic equivalence of two formulas F_1 and F_2 as:

$$[[F_1]] \equiv [[F_2]]$$

which means that given the same valuation, formulas F_1 and F_2 evaluate to the same value. For example, consider the two formulas F_1 and F_2:

$$F_1 : a \times x + b = 0 \qquad F_2 : a \times x + a + b = a$$

Note that although F_1 and F_2 are syntactically different formulas they are semantically equivalent (i.e., $F_1 \neq F_2$ and $[[F_1]] \equiv [[F_2]]$).

Programs as Functions. In this paper, we are focusing on programs that can be modeled as functions. For a given input, we assume that each execution of a program terminates and returns the same output. Formally:

Definition 3. *A* **program** C *is a total function from the domain of inputs to the domain of outputs,* $C : I \to O$*, where* $C(i) = o$ *denotes that on input* $i \in I$*, the output of* C *is* $o \in O$.

Program Equivalence. We define equivalence of programs based on their input-output behavior:

Definition 4. *Given two programs* $C_1 : I_1 \to O_1$ *and* $C_2 : I_2 \to O_2$ *where* $I_1 = I_2$*,*

- C_1 *and* C_2 *are* **equivalent**, *denoted as* $[[C_1]] \equiv [[C_2]]$*, if and only if,* $\forall i \in I, C_1(i) = C_2(i)$.
- C_1 *and* C_2 *are* **non-equivalent**, *denoted as* $[[C_1]] \not\equiv [[C_2]]$*, if and only if,* $\exists i \in I, C_1(i) \neq C_2(i)$.

Note that different implementations of the same functionality are considered equivalent according to this definition as long as the input-output behavior is the same.

Syntactic Robustness. We can now begin to define syntactic robustness for LLM-based code generators:

Definition 5. *An LLM-based code generator G is* **syntactically robust**, *if and only if, given any two prompts $P\langle T, F_1\rangle, P\langle T, F_2\rangle \in \mathcal{P}$ where $[[F_1]] \equiv [[F_2]]$, $[[G(P\langle T, F_1\rangle)]] \equiv [[G(P\langle T, F_2\rangle)]]$.*

i.e., an LLM-based code generator is syntactically robust if it generates equivalent code for semantically equivalent but syntactically different prompts.

There are two issues with the above definition that we will address. First, according to the definition, if an LLM-based code generator generates the same code for all prompts, it would be syntactically robust. i.e., an LLM-based code generator that for all prompts generates the same trivial code:

```
int main() { printf("0\n"); return 0; }
```

would be syntactically robust. To address this problem, we introduce the concept of a reference code for each prompt as follows:

Definition 6. *Given a prompt $P\langle T, F\rangle$ we call $R(P\langle T, F\rangle) = C_F^R$ the* **reference code** *for the prompt $P\langle T, F\rangle$, where C_F^R is a correct implementation of the requirements specified in the prompt $P\langle T, F\rangle$.*

Note that C_F^R can be written manually or can be generated by a code generator and validated by different means (such as manual inspection, testing, or verification).

Second, Definition 5 of syntactic robustness requires the LLM-based code generator to generate semantically equivalent programs for all semantically equivalent prompts. Even if the code generator generates semantically different code for only one syntactically different prompt while generating semantically equivalent code for all other prompts, it is not syntactically robust according to the above definition. So, we extend our definition below to measure the *syntactic robustness degree*, where Definition 5 corresponds to the highest degree of syntactic robustness. Syntactic robustness degree for an LLM-based code generator for a given prompt, its reference code, and its syntactic variations as follows:

Definition 7. *Given an LLM-based code generator G, a prompt $P\langle T, F\rangle$, a reference code $R(P\langle T, F\rangle) = C_F^R$ for prompt P, and a set of formulas $\mathcal{F}_F$ containing syntactic variations of F where for each $F' \in \mathcal{F}_F$, $[[F']] \equiv [[F]]$, let $\mathcal{F}_F^{eq} \subseteq \mathcal{F}_F$ denote the set of formulas such that for each $F' \in \mathcal{F}_F^{eq}$, $[[G(P\langle T, F'\rangle)]] \equiv [[C_F^R]]$. Then, the* **syntactic robustness degree** *of G with respect to P and $\mathcal{F}_{\mathcal{F}}$ is defined as:*

$$|\mathcal{F}_F^{eq}|/|\mathcal{F}_F|$$

where $|\mathcal{F}_F^{eq}|$ denotes the number of formulas in $\mathcal{F}_F^{eq}$ and $|\mathcal{F}_F|$ denotes the number of formulas in $\mathcal{F}_F$.

We report the syntactic robustness degree as a percentage where 100% corresponds to the case where $\mathcal{F}_F^{eq} = \mathcal{F}_F$. Note that the definition of syntactic robustness given in Definition 5 corresponds to the syntactic robustness degree of 100% for all prompts and their corresponding semantically equivalent syntactic variations.

4 Syntactic Mutations of Formulas

In this paper, we consider prompts that include mathematical equations and expressions which can be univariate or multivariate.

$$F \rightarrow E = E \mid E$$
$$E \rightarrow N \mid S \mid V \mid U \mid E + E \mid E - E \mid E \times E \mid E/E \mid E\hat{}E \mid (E)$$
$$U \rightarrow sin(E) \mid cos(E) \mid tan(E) \mid log(E) \mid ln(E) \mid V(E) \mid V'(V)$$
$$N \rightarrow -N \mid [0-9]^+ \mid [0-9]^+.\,[0-9]^+$$
$$S \rightarrow a \mid b \mid c \mid d \mid e$$
$$V \rightarrow x \mid y \mid z$$

Figure 3: Context-free grammar for mathematical formulas.

Table 1: A representative subset of mutation rules for equations and expressions (where $I \rightarrow N \mid S$ and $0 \notin N$).

Equation		Expression	
M_1	$\frac{E_1=E_2}{E_1/I=E_2/I}$	$M_{1,2}$	$\frac{E}{E\times I/I}$
M_2	$\frac{E_1=E_2}{E_1\times I=E_2\times I}$		
M_3	$\frac{E_1=E_2}{E_1+I=E_2+I}$	$M_{3,4}$	$\frac{E}{E+I-I}$
M_4	$\frac{E_1=E_2}{E_1-I=E_2-I}$		

Grammar for formulas. The context-free grammar shown in Figure 3 captures the mathematical formulas we use, where F denotes the start symbol which can be extended to either an equation ($E = E$) or an expression (E), U denotes unary functions, N denotes number literals, S denotes coefficients, and V represents the variables or functions. Note that, $V_1'(V_2)$ denotes the first derivative of function V_1 with respect to V_2.

Now, we discuss how we generate the set of formulas $\mathcal{F}_F$ that are syntactic variations of a given formula F.

Definition 8. *A* **syntactic transformation** *ST is a function that maps a formula to another formula that is semantically equivalent, i.e., $ST : \mathcal{F} \rightarrow \mathcal{F}$ such that for any formula $F \in \mathcal{F}$, $[[F]] \equiv [[ST(F)]]$, where $\mathcal{F}$ denotes the set of formulas.*

We define 18 mutation types for equations and 12 mutation types for expressions that modify syntax while preserving the semantics of the mathematical formula. These mutations include commutative addition and multiplication, adding zero, multiplication by one, and variable renaming. Table 1 presents a representative subset of the mutation rules for both expressions and equations, while the complete set is provided in the artifact [35].

Given a formula F, we generate the set of syntactic mutations of F, called $\mathcal{F}_F$ (as described in Definition 7) by repeatedly applying mutations ($M(F), M(M(F))$, $M(M(M(F)))$,...) to F, i.e., each $F' \in \mathcal{F}_F$ and by definitions of the mutations we introduced, $[[F']] \equiv [[F]]$.

Definition 9. *Given a formula F and its syntactic mutant F', the* **syntactic distance** *of F and F' is n when $F' = M^n(F)$.*

I.e., the syntactic distance between F and its mutant F' is the number of mutations needed to mutate F to F'.

5 Attacks Targeting Syntactic Robustness

We consider a black-box adversary aiming to reduce the syntactic robustness of an LLM in code generation tasks. The adversary can only apply semantics-preserving syntactic transformations to the input prompts and has no access to the model architecture or parameters. The goal is to induce incorrect or non-compiling code without changing the intended semantics. We assume the

Algorithm 1 PROFILING($\mathcal{M}$, $\mathcal{P}$)

▷ Decides effectiveness of each mutation for attack.
▷ Calls function THRESHOLD to choose a threshold to get top K mutations.

Input: $\mathcal{M}$: mutation set and $\mathcal{P}$: set of prompts to mutate.
Output: $\mathcal{W}$: set of weights describing effect of each mutation type on syntactic robustness, $\mathcal{M}_{Top-K}$: A set of Top-K mutations.

for $M \in \mathcal{M}$ **do**
  $robust_score \leftarrow \Sigma_{P \in \mathcal{P}} (|\mathcal{F}_F^{eq}|/|\mathcal{F}_F| - |\mathcal{F}_{M(F)}^{eq}|/|\mathcal{F}_{M(F)}|)$
  $\mathcal{W}[M] \leftarrow robust_score/|\mathcal{M}|$
$t_k \leftarrow$THRESHOLD($\mathcal{W}, k$)
for $M \in \mathcal{M}$ **do**
  if $\mathcal{W}[M] > t_k$ **then**
    $\mathcal{M}_{Top-K} \leftarrow \mathcal{M}_{Top-K} \cup \{M\}$
return $\mathcal{W}$, $\mathcal{M}_{Top-K}$

Algorithm 2 ATTACK(F, D, $\mathcal{M}$)

▷ Attacks by mutating a formula F to distance D.
▷ Calls function RANDOM which chooses mutations randomly based on the given weights.

Input: F: formula, D: Distance (number of mutations to apply), $\mathcal{M}$: set of mutations
Output: F': mutated formula.

$F' \leftarrow F$
for D **do**
  if Uniform **then**
    $M \leftarrow$ RANDOM($\mathcal{M}$, $\left[\frac{1}{|\mathcal{M}|} \cdots \frac{1}{|\mathcal{M}|}\right]$)
  else if Weighted **then**
    $M \leftarrow$ RANDOM($\mathcal{M}, \mathcal{W}$)
  else if Top-K **then**
    $M \leftarrow$ RANDOM($\mathcal{M}_{Top-K}$, $\left[\frac{1}{k} \cdots \frac{1}{k}\right]$)
  $F' \leftarrow M(F')$
return F'

model is pre-trained and do not consider attacks on training data or deployment-stage injection. This threat model enables systematic analysis of LLM failure modes via adversarial robustness testing [44, 45] and is applicable to automated code generation pipelines, where inputs are fed directly to LLM-based code generators [46] with minimal human oversight.

We use Algorithm 1 to profile the impact of different mutations on syntactic robustness degree of LLMs. This algorithm assigns a score ($\mathcal{W}[M]$) for each mutation (M) based on the decrease in syntactic robustness from the original prompt ($P\langle T, F\rangle$) to a modified prompt ($P\langle T, M(F)\rangle$). Based on this profiling information, we also compute a threshold (t_k) which we use to choose a subset of the mutations that have the most impact on robustness ($\mathcal{M}_{Top-K}$).

Algorithm 2 outlines our three attack strategies. Each strategy takes as input a formula F from a prompt and a syntactic distance D, and produces a mutated formula $F' = M^D(F)$. For these strategies, we take care to ensure that a formula at distance D cannot be created by a series of mutations less than D. We construct the mutation space using a dynamic programming approach: We iteratively generate all unique formulas at distance 1, then distance 2 (excluding those already generated at smaller distances), and so on up to distance D. We additionally introduce another attack strategy **NaiveUniform** which does not include this distance assurance—a mutation of distance D with the **NaiveUniform** attack could potentially be created with a lower distance.

NaiveUniform chooses randomly from mutation set, but can choose mutations that lead to a lower distance mutated formula. The **Uniform** strategy in Algorithm 2 selects a mutation uniformly at random from the set of all mutations. The **Weighted** strategy chooses mutations probabilistically with the weights calculated in PROFILING based on each mutation's impact. Lastly, **Top-K** one selects a mutation randomly from top-k most impactful mutation candidates, $\mathcal{M}_{Top-K}$, which are identified by PROFILING. For these three attack strategies in Algorithm 2, when we declare a formula F' to have distance D from F, we assure that D is the shortest distance of mutations that can produce F'.

Table 2: A representative subset of reduction rules for equations (where $I \rightarrow N \mid S$ and $0 \notin N$)

R_1	$\frac{E_1=E_2}{E_1-E_2=0}$ Shift to L.H.S.
R_2	$\frac{E+I-I}{E}$; $\frac{E_1+I+E_2-I=0}{E_1+E_2=0}$; $\frac{E_1+I-E_2-I=0}{E_1-E_2=0}$ Remove redundant addition
R_3	$\frac{E-I+I}{E}$; $\frac{E_1-I+E_2+I=0}{E_1+E_2=0}$; $\frac{E_1-I-E_2+I=0}{E_1-E_2=0}$ Remove redundant subtraction
R_4	$\frac{E\times I=0}{E=0}$; $\frac{E_1\times I+E_2\times I=0}{E_1+E_2=0}$; $\frac{E_1\times I-E_2\times I=0}{E_1-E_2=0}$ Remove redundant multiplication
R_5	$\frac{E/I=0}{E=0}$; $\frac{E_1/I+E_2/I=0}{E_1+E_2=0}$; $\frac{E_1/I-E_2/I=0}{E_1-E_2=0}$ Remove redundant division

6 Prompt Pre-processing with Formula Reduction

To improve syntactic robustness, we add a preprocessing step where, instead of feeding a prompt with a mathematical formula to the LLM-based code generator as-is, we generate a reduced version of that formula. Then, we feed the reduced prompt with the same English text and the reduced mathematical formula to the LLM-based code generator to generate the target code.

We focus on syntactic transformations that reduce the size of a formula, based on the assumption that shorter formulas are simpler and more likely to be handled correctly by LLM-based code generators. Accordingly, prompts with reduced formulas are expected to improve robustness against adversarial samples. We refer to these semantics-preserving syntactic transformations as *reductions*.

Table 3: A representative subset of reduction rules for expressions, given $I \rightarrow N \mid S$, where $0 \notin N$.

R_1	$\frac{E+I-I}{E}$; $\frac{E_1+I+E_2-I}{E_1+E_2}$; $\frac{E_1+I-E_2-I}{E_1-E_2}$ Remove redundant addition followed by subtraction
R_2	$\frac{E-I+I}{E}$; $\frac{E_1-I+E_2+I}{E_1+E_2}$; $\frac{E_1-I-E_2+I}{E_1-E_2}$ Remove redundant subtraction followed by addition
R_3	$\frac{E\times I/I}{E}$ Remove redundant multiplication/division

We define the size of a formula $|F|$ to be the number of terminal symbols in the formula as described in grammar shown in Figure 3.

Reductions. We formally define reductions as a type of syntactic transformations as follows:

Definition 10. *A* **reduction** *R is a syntactic transformation where for each formula $F \in \mathcal{F}$, $|R(F)| \leq |F|$.*

i.e., a reduction modifies the syntax of the formula without changing its semantics, while the size of the modified formula is either the same or less than the size of the original formula.

We define 9 types of reductions for equations and a representative subset of these reduction rules are shown in Table 2 (rest are presented in the artifact [35]). Reduction rule R_1 positions all nonzero elements of the equation on one side of the equality operator. Reductions R_2 and R_3 show a series of possible applications for removing redundant additions and subtractions—specifically, the two final rules show removal of these redundant operations when a different additive or subtractive term intercedes the redundant operations. Reductions R_4 and R_5 show a series of possible applications for removing redundant divisions and multiplications—this is to show that this removal may not necessarily be from one singular E comprising the full side of the formula but may also be from each individual additive or subtractive term on that side. For expressions, we introduce 7 reduction rules; we detail 3 in Table 3 and present the rest in the artifact [35]. Reductions R_1 and R_2 are similar to equation reductions R_2 and R_3, to remove redundant additions and subtractions. Reduction R_3 removes redundant multiplications and divisions. The other reduction rules include canceling the addition of extra 0's, multiplication by 1's, and other similar simple reductions. (the full set of rules is in the artifact [35]). Besides equation reduction R_1 (which is applied once at the beginning), all reductions are applied in a loop until the formula cannot be simplified further.

7 Implementation and Experimental Design

Figure 4 illustrates the workflow for assessing LLM-based code generators. We next describe our implementation and design.

Table 4: Summary of the code generation prompts

Prompt Types	Total	Classification 1		Classification 2	
		Expression	Equation	Reasoning	Translation
Solve	7	0	7	7	0
Evaluate	7	7	0	0	7
Differential	6	1	5	6	0
MaTT [43]	6	5	1	2	4
MATH [47]	7	2	5	6	1
Clover [20]	19	0	19	15	4
Total	52	15	37	36	16

Prompts: The prompts used in this paper are inspired from diverse scientific computing applications, such as solving differential equations for molecular dynamics simulations, as well as linear, polynomial, trigonometric, and logarithmic equations that are fundamental to techniques like the Fourier Transform and Fast Fourier Transform (FFT) [48], shape function interpolation [49], and exponential growth or decay modeling. We also incorporate a wide range of prompts from published benchmarks [43, 47], along with additional prompts inspired from contract-based programming [20]. Table 4 summarizes all prompt categories used in this paper. A *solve* prompt includes an equation and asks to generate code that takes the equation's coefficients as input and outputs the variable value that satisfies it. The *evaluate* prompts instruct the LLM to generate a program that evaluates a mathematical expression given the values of all variables and coefficients. The *differential* prompts involve manipulating differential formulas. The MATH [47] and Mathematical Topics Tree (MaTT) [43] datasets are designed to evaluate automated mathematical reasoning systems: MATH consists of challenging mathematics competition problems, while MaTT spans key topics across both pure and applied mathematics. The Clover [20] dataset, in contrast, focuses on verifiable code generation using LLMs and contains prompts with formal specifications.

Reasoning vs. Translation Prompts: We have divided our prompts into two categories: reasoning and translation prompts as discussed in Section 2. Reasoning prompts require the LLM to engage in mathematical manipulation and reasoning to generate the appropriate code, as they involve complex tasks that need to be understood and processed based on the mathematical formula. In contrast, translation prompts allow the LLM to directly utilize the mathematical formulas that are in the prompts as part of the generated code, as these formulas can be directly translated into program expressions. Out of the 52 prompts, we have 36 reasoning and 16 translation prompts.

Programming language: We chose C as the target language for our code generation prompts. C and C++ are extensively used for mathematical computing tasks in scientific computing due to their efficiency and performance in handling complex computations [50, 51]. Recent studies have utilized these languages in LLM-based code generation for tasks such as differential equations, numerical methods, and advanced computer science problems [10, 15, 16, 52].

Mutated formula generation: To measure the impact of the syntactic transformations of a formula on the generated code, we generate up to 20 mutations per syntactic distance between 1 and 5. The number of mutations generated at distance 1 is often less than 20, as we exhaust the available mutation operators before 20 mutations can be created. The distance 0 represents the original prompt, so the number of prompts for distance 0 is 1. For the 52 prompts we used, we have generated a total of about 4,000 mutated prompts in our experimentations.

Non-determinism of LLMs: LLMs are inherently non-deterministic. To address this, we experimented with various temperature, top_p, and seed configurations (where applicable) and selected the most deterministic settings, detailed in our artifact [35]. Additionally, we executed the LLM-based code generator five times per prompt to mitigate variability.

Post-processing of the LLM responses: Given a code generation prompt, the LLM responses typically contain the generated code and some English text explaining the generated code. Our implementation takes the generated response from an LLM, eliminates the non-code text, and converts it to a C file. The C files are then compiled into binaries using GCC [53], which is automated in our pipeline. Any C code with syntactic errors is considered non-equivalent with respect to our reference code.

Differential Testing and Code Equivalence: We implement reference solutions for each prompt and employed differential testing to evaluate the correctness and equivalence of the generated code. This involves generating 100 random inputs and comparing the outputs of the generated code with the reference solution.

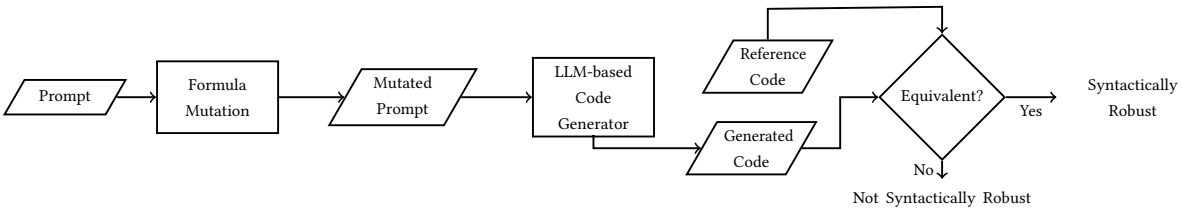


Figure 4: Assessment of Syntactic robustness degree of an LLM (this workflow is applied in a loop for multiple mutations).

Numerical inputs are randomly selected between some specified ranges based on the prompts. Additionally, our prompts require LLM-generated outputs to follow a specific format with either 6-digit or 2-digit precision, depending on the prompt type. To compare floating-point outputs, we consider them correct if they fall within a predefined epsilon threshold relative to the expected result. The epsilon threshold mitigates the impact of rounding errors, thereby relaxing our Definition 4 of program equivalence. The chosen epsilon values are detailed in our artifact [35].

When differential testing reports non-equivalence of the LLM-generated code with the reference solution, non-equivalence is guaranteed to hold. However, differential testing cannot guarantee the equivalence of LLM-generated code with the reference solution since it does not cover the whole input space. In our manual review of a subset of generated programs (100 cases), we found no instances where non-equivalent programs were falsely classified as equivalent via differential testing.

Hyperparameter for Attack Strategies: For the Top-K attack strategy, we have chosen the top 50% mutations with the most impact on the syntactic robustness degree of LLM (calculated based on Gpt-3.5). The other strategies choose the mutations randomly based on the weights (Algorithm 2).

Benchmark Models: Our evaluation includes Gpt-3.5 [54], Gpt-4o [55], Llama-3.1-70B [56] and CodeLlama [57]. Gpt-3.5 offers cost-efficiency, while Gpt-4 consistently performs well in both code generation and mathematical reasoning [58, 59]. Llama models are included for their open-source accessibility and low cost. We also evaluate o4-mini [60], a reasoning model that balances performance with reduced computational overhead compared to other OpenAI reasoning models like o3, o1-pro. We experiment with Chain-of-thought (CoT) [61] and Deeply Understanding Problems (DUP) [62] approaches as they are zero-shot prompting techniques that are generalizable, and cost-efficient. Instead of example-based CoT, we adopt trigger-based prompting (e.g., "Let's solve this step-by-step") to preserve automation [63]. DUP improved mathematical reasoning by addressing semantic misunderstadnings and has achieved state-of-the-art results on arithmetic tasks [8]; we adapt it to mathematical code generation. Regarding code generation accuracy, frameworks based on Gpt-3.5 and Gpt-4 outperform others, achieving at least 85% [8] accuracy on the HumanEval dataset [9], a trend reflected in our results. Thus, we apply framework-based models only to Gpt-based models, excluding Llama-based models due to their lower performance.

Artifact: Our artifact, including the implementation, datasets, experimental results, and the mutation and reduction rules, has been made available in a GitHub repository [35].

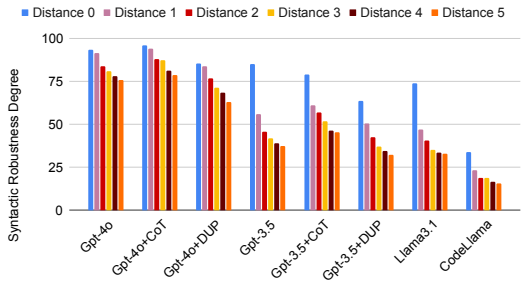


Figure 5: Syntactic Robustness Degree Vs. Distance for LLMs

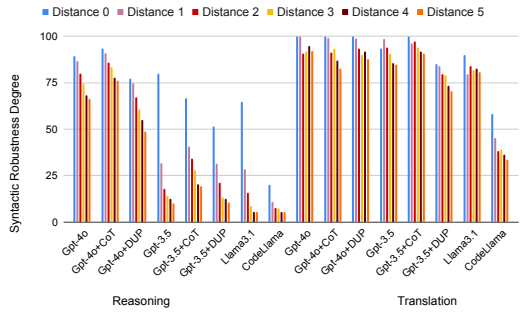


Figure 6: Syntactic robustness degree of reasoning prompts (left 8) and translation prompts (right 8)

8 Experimental Evaluation

We address four research questions:

RQ1: Assessment 1: Does LLM-based code generation achieve syntactic robustness?

RQ2: Assessment 2: Do LLMs show different levels of syntactic robustness for translation and reasoning prompts in code generation?

RQ3: Exploitation: Do the attacks help in decreasing the syntactic robustness of the LLM-based code generation?

RQ4: Mitigation: Does prompt pre-processing with formula reduction improve the syntactic robustness of code generation and help in mitigating our attacks?

Below we present the results of our experiments and discuss the four research questions. For our assessment (RQ1-2), we use the Gpt-3.5, Gpt-4o, Llama-3.1-70B, CodeLlama, Gpt-based framework models and we use the 33 prompts from Table 4 (Row 1-5). For RQ3, we have chosen Gpt-3.5 as it performs better than open source models and cheaper than other Gpt-based models. To evaluate our reduction technique (RQ4), we have extended our dataset to Clover dataset and also included o4-mini model.

RQ1: Assessment 1: Figure 5 illustrates the syntactic robustness percentages for all foundational and framework models across different syntactic distances, based on the average robustness scores across the 33 prompts used. Distance 0 shows the robustness of code generation for the original formula, while Distance 1 through Distance 5 shows the average syntactic robustness degrees for the mutated variations at each respective syntactic distance. As shown in Figure 5, the syntactic robustness degree decreases consistently across all models as syntactic distance increases. Despite varying performance levels, none of the models demonstrate complete syntactic robustness. Gpt-4o+CoT performs best, with an average syntactic robustness of 87.4%, followed by Gpt-4o with 83.87%.

Moreover, Figure 5 shows average syntactic robustness percentages across mutation distances from 0 to 5 and clearly demonstrates that as the number of mutation increases, syntactic robustness decreases across all foundational and framework-based models. Notably, the rate of decline varies among different LLMs. Gpt-4o+CoT shows the slowest decline in syntactic robustness degree per mutation distance: -3.45% on average. Gpt-3.5 experiences a more significant drop of -9.52%, as its syntactic robustness degree with the original prompt is higher than the robustness degree of mutated prompts. In summary, the syntactic robustness decreases for all models as mutation distance increases. Thus, we conclude that **LLMs do not achieve syntactic robustness** and **increasing number of mutations (equivalently, increasing syntactic distance) leads to a decrease in syntactic robustness**.

RQ2: Assessment 2: Figure 6 shows the average syntactic robustness of all reasoning prompts (left side) and translation prompts (right side) across eight models. The results indicate that translation prompts consistently exhibit higher robustness than reasoning prompts. As mutation distance increases, robustness declined more steeply for reasoning prompts. On average, reasoning prompts achieve a syntactic robustness of 42.86%, compared to 83.80% for translation prompts. The robustness drops by approximately 30% for reasoning prompts and 8% for translation ones relative to their original forms. Based on our experimental evaluation, we conclude that **LLMs exhibit different levels of syntactic robustness across translation and reasoning prompts, with greater robustness for translation prompts**.

RQ3: Exploitation: As described in RQ2, translation prompts are more robust than the reasoning prompts. The attacks on the translation prompts also show similar results (details in artifact [35]). Thus, to answer this research question, we only focus on the reasoning prompts. Figure 7 shows the impact of our different attack strategies on Gpt-3.5 model. It illustrates that for all attack strategies, increasing syntactic distance decreases the syntactic robustness degree, which follows the findings in RQ1. On average, across all distances, the syntactic robustness degree is 36.13%, 30.17%, 17.54% and 14.64% respectively for NaiveUniform, Uniform, Weighted and Top-K attack strategies. The Weighted and Top-K attack strategies are most effective in reducing syntactic robustness degree across all tested distances. In Figure 8, we show how many mutations are required for different attack strategies to reduce the syntactic robustness degree to a target level (x-axis). With NaiveUniform, 4 mutations are required to reduce syntactic robustness to 25% and more than 5 would be necessary for further reduction. On the other hand, similar syntactic robustness can be achieved with a lower

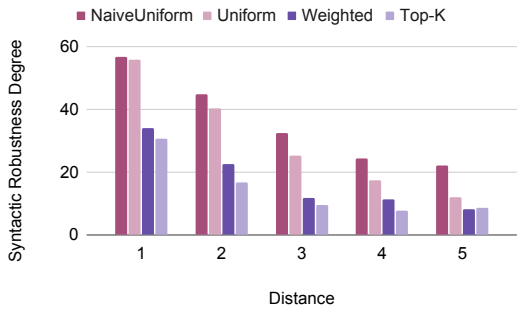


Figure 7: Syntactic robustness degree of different attacks for reasoning prompts (Gpt-3.5)

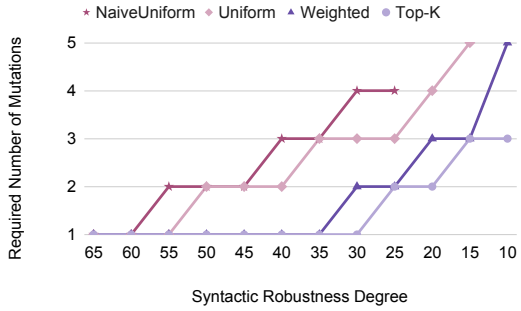


Figure 8: Required number of mutations for different attacks

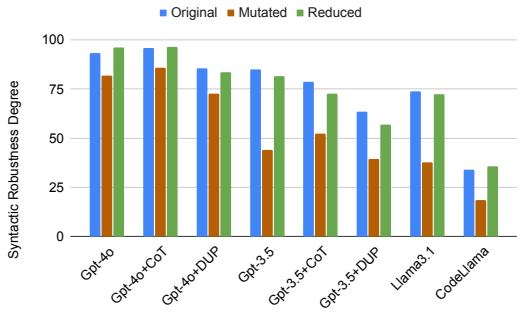


Figure 9: Syntactic robustness degree with reduced form

number of mutations for the other attacks (3 for Uniform and 2 for Weighted and Top-K). To reduce the syntactic robustness down to 10%, Weighted requires 5 mutations whereas Top-K requires only 3 mutations. Thus, Figure 8 illustrates that Top-K is most effective in getting the highest reduction in syntactic robustness with the least number of mutations. Our experimental evaluation shows that **our attack strategies are effective in decreasing the syntactic robustness degree**.

RQ4: Mitigation: We demonstrate the mitigating effect of applying our reduction rules to mutated formulas before querying the LLM-based code generator. Figure 9 presents the syntactic robustness of the models for the original formulas, the average robustness for mutated formulas, and the robustness of the reduced prompts. Across all foundational and framework models, the reduced prompts exhibit robustness degrees comparable to the original prompts and show notable improvement over the mutated

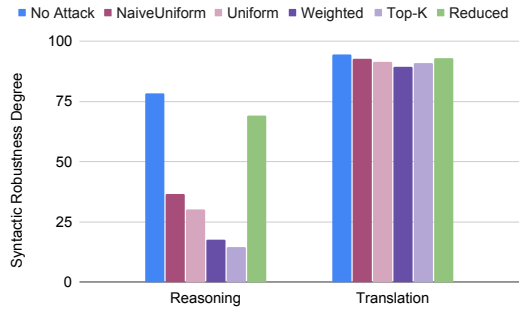


Figure 10: Mitigation of attack strategies (Gpt-3.5)

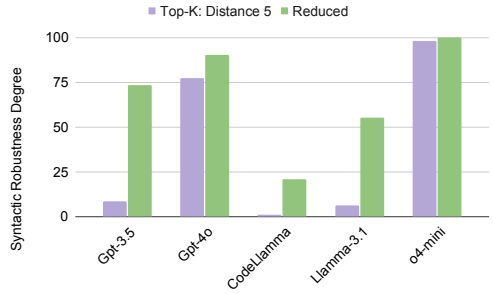


Figure 11: Impact of Top-K attacks on reasoning prompts at distance 5

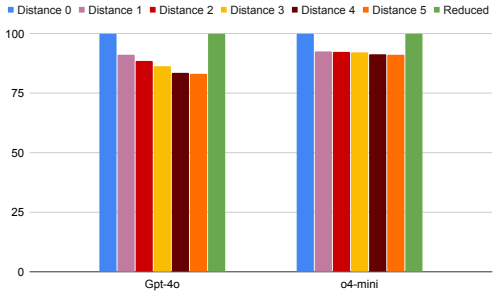


Figure 12: Top-K attack for Clover Dataset

prompts. The improvement in syntactic robustness when comparing reduced prompts to mutated prompts is substantial, with increases between 10.63% and 37.55%, resulting in an overall average improvement of 20.37%. Among all foundational and framework models, the reduced prompts for Gpt-4o+CoT perform the best on average for all the prompts we have used.

For Gpt-4o and Gpt-4o+CoT, the reduced prompts show improvements of 2.73% and 0.6%, respectively, even over the original prompts. Moreover, CodeLlama achieves an improvement of 5.1% for reduced prompts compared to the original ones. In contrast, other models show a slight degradation in performance with the reduced prompts compared to the original prompts.

Figure 10 shows the syntactic robustness degrees of different attacks along with the original and our reduced prompts for both reasoning and translation prompts in Gpt-3.5. It is evident that our pre-processing step helps in mitigating the impact of attacks, especially for reasoning prompts. Moreover, in Figure 11, we have shown the results for the Top-K attack at distance 5 and the robustness for the corresponding reduced prompts across different LLMs (including o4-mini). In all cases, reduction improves the syntactic robustness. Even though o4-mini is very robust against the attack, there is a 1.95% drop in robustness which can be mitigated using our reduction step. Figure 12 also shows the result across different distances for Top-K attack on Gpt-4o and o4-mini on Clover dataset. This result on a different dataset also follows the trend of decreasing robustness with distances which was successfully mitigated by our reduction steps. Overall, our reduction rules are effective in pre-processing the prompts for reliable code generation, and the experimental results confirm that **applying our pre-processing step to simplify formulas enhances the syntactic robustness of LLM-based code generation**.

Discussion. All foundational models were evaluated using the same prompts from our dataset. However, the framework models, such as Gpt+CoT and Gpt+DUP, require additional text in the prompts. Although this extra text does not alter the instructions about the input and the output format of the code, we observed that the framework models, particularly Gpt-3.5+DUP and Gpt-3.5+CoT, often fail to follow the given instructions. For instance, Gpt-3.5+DUP produced only the instruction to generate code in the comment section of the C program in 22 cases, while Gpt-3.5+CoT did the same in 3 cases. Additionally, in 18 cases, Gpt-3.5+DUP generated code without incorporating the input parameters, which disrupts the automated differential testing process. This issue is even more pronounced in Llama-based models, where code is generated without user input processing in approximately 150 cases for Llama-3.1-70B and 600 cases for CodeLlama. These factors negatively impact the overall performance of Llama and the framework-based models in code generation. We also observe that Llama-3.1 and CodeLlama often choose to present results in other languages such as C++ and Python despite clear instructions to use C, which lowers the syntactic robustness.

Although we selected o4-mini for its efficiency over larger reasoning models like o3 and o1-pro, our experiments reveal that it still takes approximately 4× more time than Gpt-4o and about 10× more time than Gpt-3.5 for comparable tasks.

Threats to Validity. We discuss possible internal and external threats to validity and the efforts we have made to mitigate them.

Internal: There are a few possible internal threats to validity within our experimental design. First, there is the possibility that the Gpt API (for Gpt-3.5, Gpt-4o, and o4-mini) may be learning from prior requests we send. However, we believe this is not the case, as we have switched APIs during these experiments multiple times and found no observable difference between code returned from a previously-used API and code returned from a fresh one. Second, we note that differential testing cannot prove equivalence, and due to needing an epsilon value in 31 prompts to avoid incorrectly marking nonequivalence due to floating point error, it is possible some incorrect results could be marked correct. This is a possible risk for any testing-based equivalence checking approach. However, we have manually investigated a subset of LLM generated programs and found no case where differential testing falsely marked two non-equivalent codes as equivalent. Moreover,

when we declare generated code non-equivalent, non-equivalence is guaranteed since we produce a test case that demonstrates the non-equivalence.

External: Analyzing all formulas or mutations is not possible. However, we believe that our evaluation demonstrates syntactic robustness issues for LLMs for an important class of formulas and mutations. Furthermore, our evaluation and attack approaches can be applied to different classes of formulas and syntactically equivalent representations of those formulas using the framework we present in this paper.

Limitations. Our work focuses on the robustness of LLMs specifically for code generation prompts involving mathematical formulas and constraints. Consequently, our transformation and reduction techniques are also limited to prompts containing such mathematical formulas. However, mathematical computation–based code generation is fundamental to many domains, including scientific computing and contract-based programming. While our benchmark consists of a limited number of prompts, we believe they represent a diverse set of prompts with mathematical formulas and can be further extended. Additionally, while we evaluate code generation for the C programming language, our approach can be extended to assess other LLMs for different programming languages.

9 Related Work

Prior work on LLM-based code generation has focused primarily on improving performance on software engineering tasks [9, 25, 64, 65]. Much of this research emphasizes accuracy on benchmarks such as HumanEval, MBPP, and APPS [9, 66, 67]. In parallel, substantial work studies mathematical problem solving with LLMs [62, 68, 69], evaluated using datasets such as GSM8K, SVAMP, and MultiArith [70–72]. For code generation with mathematical computation, the LLMs must be effective in mathematical reasoning as well as in generating code. Recent work in scientific computing and numerical analysis [10–13] demonstrates that these capabilities can be combined to solve complex tasks, including contract programming [20]. However, none of these studies examine the robustness of LLM-based code generators under syntactic variations, which is the focus of this work.

Robustness has been extensively studied for classification and regression neural networks, using techniques such as symbolic reasoning [26–28], abstraction [29, 30], and testing/fuzzing [31, 32].

Prior work has examined the correctness of LLMs for code generation tasks [73–80]. Other studies investigate robustness against non-determinism in LLMs [34], perturbations in natural language descriptions [5, 21], and variations in coding components [22–25, 81]. ReCode [6] introduces a robustness evaluation benchmark for code generation by perturbing prompts, including both natural language components and code segments via partial code syntax refactoring in code completion settings. Semantic preservation in ReCode relies on human annotation. In contrast, our work evaluates robustness through perturbations of mathematical formulas that are unambiguously semantics-preserving and do not require human annotation. Moreover, ReCode's prompts, transformations, and datasets do not target mathematical reasoning and do not include prompts containing mathematical formulas. Robustness has also been studied for mathematical problem solving by LLMs [44, 82–86] which are not focused on code generation by LLMs. Compared to prior work and surveys [1, 87], our work formally defines syntactic robustness and focuses on prompts containing mathematical formulas, targeting the robustness of LLM-based code generation.

Our syntactic transformation approach, which mutates mathematical formulas, is conceptually similar to metamorphic testing [88], a property-based testing technique that has also been applied to LLMs [89, 90]. For example, LAMPION [89] generates equivalent input code snippets to test robustness of LLM-based program analysis models. Unlike prior work, we target LLM-based code generation.

Recent multi-agent code generation frameworks like AgentCoder, LDB, and MapCoder [7, 91, 92] achieve over 90% accuracy on benchmarks such as HumanEval and MBPP [8], but require multiple LLM queries per prompt [93, 94]. Studies show that when both accuracy and cost are considered, these frameworks do not outperform baseline foundational models [93], and they also rely on input-output test cases [67, 95]. In contrast, our work focuses on prompt-engineering frameworks that rely solely on prompts.

Beyond multi-agent methods, significant progress has been made with prompt engineering frameworks [74, 96–106]. CoT has been applied to code generation by decomposing problems into sequential, branch, and loop structures [107, 108]. Other reasoning-oriented prompting strategies, such as DUP and AceCoder [4, 62, 82, 109–111], also achieve strong performance, with DUP attaining state-of-the-art results on arithmetic reasoning benchmarks [8]. Since our dataset combines mathematical reasoning and code generation, we evaluate both CoT and DUP.

Prior robustness attacks on deep code models [112] primarily target misclassification in tasks like authorship attribution [36, 113]. In contrast, we focus on LLM-based code generators, which fundamentally differ from classifiers. While black-box prompt attacks [44, 114] exist, they do not address code generation. Other work has explored code completion or generation attacks using adversarial comments (INSEC [115]) or non-functional code changes [45]. Our approach instead mutates mathematical formulas to produce semantically equivalent prompts, exposing failures in syntactic robustness.

There are also works which focus on training strategies or architectural changes for improving the robustness of LLM code generations [116–119]. Our approach is distinct as we focus on black-box prompt pre-processing for improving this robustness, without altering or retraining the models.

10 Conclusion

The use of LLM-based code generation is expanding in domains involving mathematical computations. In this paper, we expose limitations of state-of-the-art LLMs when handling prompts with mathematical formulas under semantic-preserving syntactic mutations. We formalize this concept as syntactic robustness and evaluate multiple benchmark models using both original and mutated prompts. Our results show a significant decrease in robustness as syntactic distance increases, with reasoning-intensive prompts particularly affected. We further design and compare attack strategies that effectively reduce robustness. To mitigate this issue, we introduce a formula reduction-based prompt pre-processing technique that consistently improves robustness across all evaluated models.

References

[1] A. Fan, B. Gokkaya *et al.*, "Large language models for software engineering: Survey and open problems," in *IEEE/ACM International Conference on Software Engineering: Future of Software Engineering, ICSE-FoSE*. IEEE, 2023, pp. 31–53.

[2] X. Hou, Y. Zhao *et al.*, "Large language models for software engineering: A systematic literature review," *ACM Trans. Softw. Eng. Methodol.*, 2024.

[3] J. Jiang, F. Wang *et al.*, "A survey on large language models for code generation," 2024. [Online]. Available: https://arxiv.org/abs/2406.00515

[4] X. Jiang, Y. Dong *et al.*, "Self-planning code generation with large language models," *ACM Transactions on Software Engineering and Methodology*, 2023.

[5] A. Mastropaolo, L. Pascarella *et al.*, "On the robustness of code generation techniques: An empirical study on github copilot," in *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023*. IEEE, 2023, pp. 2149–2160.

[6] S. Wang, Z. Li *et al.*, "ReCode: Robustness evaluation of code generation models," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, 2023.

[7] D. Huang, Q. Bu *et al.*, "Agentcoder: Multi-agent-based code generation with iterative testing and optimisation," *arXiv:2312.13010*, 2023.

[8] "State of the art code generation benchmarks," https://paperswithcode.com/task/code-generation, accessed: 2025-11-04.

[9] M. Chen, J. Tworek *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[10] A. Kashefi and T. Mukerji, "Chatgpt for programming numerical methods," *Journal of Machine Learning for Modeling and Computing*, vol. 4, no. 2, 2023.

[11] M. R. AI4Science and M. A. Quantum, "The impact of large language models on scientific discovery: a preliminary study using gpt-4," 2023. [Online]. Available: https://arxiv.org/abs/2311.07361

[12] P. Shojaee, K. Meidani *et al.*, "LLM-SR: scientific equation discovery via programming with large language models," in *The Thirteenth International Conference on Learning Representations, ICLR 2025*, 2025.

[13] M. Du, Y. Chen *et al.*, "Llm4ed: Large language models for automatic equation discovery," *arXiv:2405.07761*, 2024.

[14] S. Koceski, N. Koceska *et al.*, "Can chatgpt be used for solving ordinary differential equations," *International Electronic Journal of Mathematics Education*, vol. 6, 12 2023. [Online]. Available: https://eprints.ugd.edu.mk/32896/

[15] S. Koceski, N. Koceska, L. Koceva Lazarova, M. Miteva, and B. Zlatanovska, "Using chatgpt for numerical solution of first and second order ordinary differential equations," 6 2023, presentation at the 10th Jubilee International Conference of FMNS (FMNS-2023), Blagoevgrad, Bulgaria. [Online]. Available: https://eprints.ugd.edu.mk/32144/1/FMNS_2023_Bulgaria%20Chat%20GPT.pdf

[16] G. Orlando, "Assessing chatgpt for coding finite element methods," *Journal of Machine Learning for Modeling and Computing*, vol. 4, 07 2023.

[17] S. Vishnu, Sahil, and N. Garg, "Unveiling the role of gpt-4 in solving leetcode programming problems," *Computer Applications in Engineering Education*, vol. 33, no. 1, p. e22815, 2025.

[18] T. Coignion, C. Quinton, and R. Rouvoy, "A performance study of llm-generated code on leetcode," in *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*, 2024, pp. 79–89.

[19] Y. Li, J. Parsert, and E. Polgreen, "Guiding enumerative program synthesis with large language models," in *Computer Aided Verification*, A. Gurfinkel and V. Ganesh, Eds. Cham: Springer Nature Switzerland, 2024, pp. 280–301.

[20] C. Sun, Y. Sheng, O. Padon, and C. Barrett, "Clover: Closed-loop verifiable code generation," in *AI Verification*, G. Avni, M. Giacobbe, T. T. Johnson, G. Katz, A. Lukina, N. Narodytska, and C. Schilling, Eds. Cham: Springer Nature Switzerland, 2024, pp. 134–155.

[21] E. Jiang, E. Toh *et al.*, "Discovering the syntax and strategies of natural language programming with generative language models," in *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*, 2022, pp. 1–19.

[22] M. Yan, J. Chen *et al.*, "Coco: Testing code generation systems via concretized instructions," *arXiv:2308.13319*, 2023.

[23] L. Zhong and Z. Wang, "Can llm replace stack overflow? a study on robustness and reliability of large language model code generation," *Proceedings of the AAAI Conference on Artificial Intelligence*, 03 2024.

[24] A. Buscemi, "A comparative study of code generation using chatgpt 3.5 across 10 programming languages," *arXiv:2308.04477*, 2023.

[25] F. F. Xu, U. Alon, G. Neubig, and V. J. Hellendoorn, "A systematic evaluation of large language models of code," in *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*, 2022, pp. 1–10.

[26] G. Katz, D. A. Huang *et al.*, "The marabou framework for verification and analysis of deep neural networks," in *International Conference on Computer Aided Verification*, 2019.

[27] R. Bunel, P. Mudigonda *et al.*, "Branch and bound for piecewise linear neural network verification," *Journal of Machine Learning Research*, vol. 21, 2020.

[28] Y. Sun, M. Wu *et al.*, "Concolic testing for deep neural networks," in *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*, 2018, pp. 109–119.

[29] G. Singh, T. Gehr, M. Püschel, and M. Vechev, "An abstract domain for certifying neural networks," *Proceedings of the ACM on Programming Languages*, vol. 3, no. POPL, pp. 1–30, 2019.

[30] S. Wang, K. Pei *et al.*, "Formal security analysis of neural networks using symbolic intervals," in *27th {USENIX} Security Symposium ({USENIX} Security 18)*, 2018.

[31] X. Xie, L. Ma *et al.*, "Deephunter: a coverage-guided fuzz testing framework for deep neural networks," in *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2019, pp. 146–157.

[32] T. Baluta, Z. L. Chua, K. S. Meel, and P. Saxena, "Scalable quantitative verification for deep neural networks," in *IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 2021.

[33] T. Gehr, M. Mirman *et al.*, "Ai2: Safety and robustness certification of neural networks with abstract interpretation," in *Symposium on Security and Privacy (SP)*. IEEE, 2018.

[34] S. Ouyang, J. Zhang *et al.*, "An empirical study of the non-determinism of chatgpt in code generation," *ACM Trans. Softw. Eng. Methodol.*, 01 2025.

[35] https://github.com/laboni68/LLMEvaluation.git, 2025.

[36] Q. Liu, S. Ji, C. Liu, and C. Wu, "A practical black-box attack on source code authorship identification classifiers," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 3620–3633, 2021.

[37] M. Hort, L. Vidziunas, and L. Moonen, "Semantic-preserving transformations as mutation operators: A study on their effectiveness in defect detection," in *2025 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. IEEE, 2025, pp. 337–346.

[38] Z. Tian, J. Chen, and Z. Jin, "Code difference guided adversarial example generation for deep code models," in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2023, pp. 850–862.

[39] K. Zhu, J. Wang *et al.*, "Promptrobust: Towards evaluating the robustness of large language models on adversarial prompts," in *Proceedings of the 1st ACM Workshop on Large AI Systems and Models with Privacy and Safety Analysis*, 2023.

[40] X. Ding, Y. Huang, X. Chen, and J. Bian, "Adversarial attack and robustness improvement on code summarization," in *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*, 2024, pp. 17–27.

[41] H. Zhang, Z. Fu *et al.*, "Towards robustness of deep program processing models – detection, estimation and enhancement," *ACM Transactions on Software Engineering and Methodology*, vol. 31, 01 2022.

[42] "Hackerrank," https://www.hackerrank.com/contests/projecteuler/challenges, accessed: 2025-11-04.

[43] A. G. Davoodi, S. P. M. Davoudi, and P. Pezeshkpour, "Llms are not intelligent thinkers: Introducing mathematical topic tree benchmark for comprehensive evaluation of llms," in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL 2025*.

[44] Z. Zhou, Q. Wang *et al.*, "Mathattack: Attacking large language models towards math solving ability," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, 2024.

[45] Q. Peng, C. Zhang *et al.*, "Random perturbation attack on llms for code generation," in *4th IEEE/ACM International Conference on AI Engineering - Software Engineering for AI, CAIN*, 2025.

[46] K. Zhang, J. Li *et al.*, "CodeAgent: Enhancing code generation with tool-integrated agent systems for real-world repo-level coding challenges," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.

[47] D. Hendrycks, C. Burns *et al.*, "Measuring mathematical problem solving with the math dataset," *35th Conference on Neural Information Processing Systems (NeurIPS 2021) Track on Datasets and Benchmarks*, 03 2021.

[48] P. S. Heckbert, "Fourier transforms and the fast fourier transform (fft) algorithm," 1998. [Online]. Available: https://api.semanticscholar.org/CorpusID:6022157

[49] T. G. Zieli, "Introduction to finite element method," 1992.

[50] A. Iserles, "Numerical recipes in c—the art of scientific computing, by wh press, bp flannery, sa teukolsky and wt vetterling. pp 735.£ 27· 50. 1988. isbn 0-521-35465-x (cambridge university press)," *The Mathematical Gazette*, 1989.

[51] G. Hager and G. Wellein, *Introduction to High Performance Computing for Scientists and Engineers*, 1st ed. USA: CRC Press, Inc., 2010.

[52] E. Catir, R. Claesson, and R. M. Tsoupidi, "Evaluating code generation of llms in advanced computer science problems," *arXiv preprint arXiv:2504.14964*, 2025.

[53] "gcc," https://gcc.gnu.org/, accessed: 2025-11-04.

[54] "Gpt-3.5 turbo," https://platform.openai.com/docs/models/gpt-3-5-turbo, accessed: 2025-11-04.

[55] OpenAI, J. Achiam, S. Adler *et al.*, "Gpt-4 technical report," 2024.

[56] A. Dubey, A. Jauhri *et al.*, "The llama 3 herd of models," *arXiv:2407.21783*, 2024.

[57] B. Roziere, J. Gehring *et al.*, "Code llama: Open foundation models for code," *arXiv:2308.12950*, 2023.

[58] Z. Wang, L. Zhang *et al.*, "How does naming affect llms on code analysis tasks?" 2024. [Online]. Available: https://arxiv.org/abs/2307.12488

[59] J. Ahn, R. Verma *et al.*, "Large language models for mathematical reasoning: Progresses and challenges," in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: Student Research*

Workshop, Mar. 2024.
[60] "o4-mini," https://platform.openai.com/docs/models/o4-mini, accessed: 2025-05-30.
[61] J. Wei, X. Wang, D. Schuurmans, M. Bosma, Xia *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," *Advances in neural information processing systems*, vol. 35, 2022.
[62] Q. Zhong, K. Wang *et al.*, "Achieving >97% on GSM8K: deeply understanding the problems makes llms better solvers for math word problems," *Frontiers Comput. Sci.*, 2026.
[63] T. Kojima, S. S. Gu *et al.*, "Large language models are zero-shot reasoners," *Advances in neural information processing systems*, 2022.
[64] Y. Li, D. Choi *et al.*, "Competition-level code generation with alphacode," *Science*, vol. 378, no. 6624, pp. 1092–1097, 2022.
[65] E. Nijkamp, B. Pang *et al.*, "Codegen: An open large language model for code with multi-turn program synthesis," in *The Eleventh International Conference on Learning Representations, ICLR*, 2023.
[66] J. Austin, A. Odena *et al.*, "Program synthesis with large language models," *arXiv:2108.07732*, 2021.
[67] D. Hendrycks, S. Basart *et al.*, "Measuring coding challenge competence with APPS," in *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks*, 2021.
[68] J. He-Yueya, G. Poesia *et al.*, "Solving math word problems by combining language models with symbolic solvers," *arXiv:2304.09102*, 2023.
[69] J. Uesato, N. Kushman *et al.*, "Solving math word problems with process-and outcome-based feedback," *arXiv:2211.14275*, 2022.
[70] K. Cobbe, V. Kosaraju, *et al.*, "Training verifiers to solve math word problems," *arXiv:2110.14168*, 2021.
[71] A. Patel, S. Bhattamishra, and N. Goyal, "Are NLP models really able to solve simple math word problems?" in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT*, 2021.
[72] S. Roy and D. Roth, "Solving general arithmetic word problems," in *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing, EMNLP*, 2015.
[73] B. Yetiştiren, I. Özsoy *et al.*, "Evaluating the code quality of ai-assisted code generation tools: An empirical study on github copilot, amazon codewhisperer, and chatgpt," *arXiv.2304.10778*, 04 2023.
[74] T. Ahmed, K. S. Pai, P. Devanbu, and E. T. Barr, "Improving few-shot prompts with relevant static analysis products," *arXiv:2304.06815*, 2023.
[75] J. Liu, C. S. Xia *et al.*, "Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation," *Advances in Neural Information Processing Systems*, 2024.
[76] A. Borji, "A categorical archive of chatgpt failures," *arXiv:2302.03494*, 2023.
[77] T. Dinh, J. Zhao *et al.*, "Large language models of code fail at completing code with potential bugs," *Advances in Neural Information Processing Systems*, 2024.
[78] C. Spiess, D. Gros *et al.*, "Calibration and correctness of language models for code," in *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering*, 2025.
[79] F. He, J. Zhai, and M. Pan, "Beyond code generation: Assessing code llm maturity with postconditions," *arXiv:2407.14118*, 2024.
[80] K. D. Wang, E. Burkholder *et al.*, "Examining the potential and pitfalls of chatgpt in science and engineering problem-solving," in *Frontiers in Education*, vol. 8. Frontiers Media SA, 2024.
[81] F. Rabbi, Z. Ding, and J. Yang, "A multi-language perspective on the robustness of llm code generation," *arXiv preprint arXiv:2504.19108*, 2025.
[82] U. Anantheswaran, H. Gupta *et al.*, "Investigating the robustness of llms on math word problems," *arXiv:2406.15444*, 2024.
[83] Q. Li, L. Cui, X. Zhao, L. Kong, and W. Bi, "Gsm-plus: A comprehensive benchmark for evaluating the robustness of llms as mathematical problem solvers," *arXiv:2402.19255*, 2024.
[84] V. Kumar, R. Maheshwary, and V. Pudi, "Adversarial examples for evaluating math word problem solvers," in *Findings of the Association for Computational Linguistics: EMNLP 2021*, 2021.
[85] B. Zhang, C. Li, and K. Fan, "Mario eval: Evaluate your math llm with your math llm–a mathematical dataset evaluation toolkit," *arXiv:2404.13925*, 2024.
[86] Y. Hao, X. Wan, and C. Zhai, "An investigation of robustness of llms in mathematical reasoning: Benchmarking with mathematically-equivalent transformation of advanced mathematical problems," *arXiv preprint arXiv:2508.08833*, 2025.
[87] Z. Yang, Z. Sun, T. Y. Zhuo, P. Devanbu, and D. Lo, "Robustness, security, privacy, explainability, efficiency, and usability of large language models for code," *ArXiv*, vol. abs/2403.07506, 2024.
[88] T. Y. Chen, S. C. Cheung, and S. M. Yiu, "Metamorphic testing: a new approach for generating next test cases," *arXiv:2002.12543*, 2020.
[89] L. Applis, A. Panichella, and A. van Deursen, "Assessing robustness of ml-based program analysis tools using metamorphic program transformations," in *International Conference on Automated Software Engineering (ASE)*. IEEE, 2021.
[90] L. Applis, A. Panichella, and R. Marang, "Searching for quality: Genetic algorithms and metamorphic testing for software engineering ml," in *Proceedings of the Genetic and Evolutionary Computation Conference*, 2023.
[91] L. Zhong, Z. Wang, and J. Shang, "Ldb: A large language model debugger via verifying runtime execution step-by-step," *arXiv:2402.16906*, 2024.
[92] M. A. Islam, M. E. Ali, and M. R. Parvez, "Mapcoder: Multi-agent code generation for competitive problem solving," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics ACL 2024*. Association for Computational Linguistics, 2024.
[93] S. Kapoor, B. Stroebl *et al.*, "AI agents that matter," *Trans. Mach. Learn. Res.*, 2025.
[94] "Ai leaderboards are no longer useful," https://www.aisnakeoil.com/p/ai-leaderboards-are-no-longer-useful#footnote-3-144156093, accessed: 2025-11-04.
[95] M. A. M. Khan, M. S. Bari *et al.*, "Xcodeeval: An execution-based large scale multilingual multitask benchmark for code understanding, generation, translation and retrieval," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.
[96] C. Liu, B. Xuanlin, H. Zhang, N. Zhang, H. Hu, X. Zhang, and M. Yan, "Improving chatgpt prompt for code generation," *arXiv preprint arXiv:2305.08360*, 05 2023.
[97] G. Marvin, N. Hellen, D. Jjingo, and J. Nakatumba-Nabende, "Prompt engineering in large language models," in *International conference on data intelligence and cognitive informatics*. Springer, 2023.
[98] J. Li, Y. Zhao, Y. Li, G. Li, and Z. Jin, "Towards enhancing in-context learning for code generation," *arXiv:2303.17780*, 2023.
[99] J. Döderlein, N. H. Kouadio *et al.*, "Piloting copilot, codex, and starcoder2: Hot temperature, cold prompts, or black magic?" *J. Syst. Softw.*, 2025.
[100] J. He and M. Vechev, "Controlling large language models to generate secure and vulnerable code," *arXiv e-prints*, 2023.
[101] J. White, S. Hays, Q. Fu, J. Spencer-Smith, and D. C. Schmidt, "Chatgpt prompt patterns for improving code quality, refactoring, requirements elicitation, and software design," *arXiv:2303.07839*, 2023.
[102] J. Li, Y. Li *et al.*, "Skcoder: A sketch-based approach for automatic code generation," in *ACM 45th International Conference on Software Engineering (ICSE)*, 2023.
[103] J. Li, G. Li, Y. Li, and Z. Jin, "Enabling programming thinking in large language models toward code generation," *arXiv:2305.06599*, 2023.
[104] S. Jiang, Y. Wang, and Y. Wang, "Selfevolve: A code evolution framework via large language models," *arXiv:2306.02907*, 2023.
[105] K. Zhang, Z. Li *et al.*, "Self-edit: Fault-aware code editor for code generation," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics ACL 2023*. Association for Computational Linguistics, 2023.
[106] F. Mu, L. Shi *et al.*, "Clarifygpt: A framework for enhancing llm-based code generation via requirements clarification," *Proceedings of the ACM on Software Engineering*, no. FSE, 2024.
[107] Z. Shao, Y. Gong *et al.*, "Synthetic prompting: Generating chain-of-thought demonstrations for large language models," in *International Conference on Machine Learning*. PMLR, 2023.
[108] J. Li, G. Li, Y. Li, and Z. Jin, "Structured chain-of-thought prompting for code generation," *ACM Transactions on Software Engineering and Methodology*, 2023.
[109] J. Li, Y. Zhao *et al.*, "Acecoder: An effective prompting technique specialized in code generation," *ACM Trans. Softw. Eng. Methodol.*, 2024.
[110] A. Didolkar, A. Goyal *et al.*, "Metacognitive capabilities of llms: An exploration in mathematical problem solving," in *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems, NeurIPS*, 2024.
[111] Y. Wu, F. Jia *et al.*, "Mathchat: Converse to tackle challenging math problems with llm agents," in *ICLR 2024 Workshop on Large Language Model (LLM) Agents*, 2024.
[112] Y. Qu, S. Huang, and Y. Yao, "A survey on robustness attacks for deep code models," *Automated Software Engineering*, vol. 31, p. 65, 2024.
[113] F. Gao, Y. Wang, and K. Wang, "Discrete adversarial attack to models of code," *Proceedings of the ACM on Programming Languages (PLDI)*, 2023.
[114] X. Xu, K. Kong *et al.*, "An LLM can fool itself: A prompt-based adversarial attack," in *The Twelfth International Conference on Learning Representations, ICLR*, 2024.
[115] S. Jenko, J. He, N. Mündler, M. Vero, and M. Vechev, "Black-box adversarial attacks on llm-based code completion," *arXiv preprint arXiv:2408.02509*, 2024.
[116] Y. Ding, J. Peng *et al.*, "Semcoder: Training code language models with comprehensive semantics reasoning," in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
[117] S. Chakraborty, T. Ahmed *et al.*, "Natgen: generative pre-training by "naturalizing" source code," in *Proceedings of the 30th ACM joint european software engineering conference and symposium on the foundations of software engineering*, 2022, pp. 18–30.
[118] K. Pei, W. Li, Q. Jin, S. Liu, S. Geng, L. Cavallaro, J. Yang, and S. Jana, "Exploiting code symmetries for learning program semantics," in *Forty-first International Conference on Machine Learning*, 2024.
[119] Y. Zhang, S. Wang *et al.*, "Codefort: Robust training for code generation models," in *Findings of the Association for Computational Linguistics: EMNLP*, 2024.